GenAI RAG System

Technical Documentation

High-Level & Low-Level Architecture

EY Internship 2026 — Updated Edition

Field	Value
Author	Likhith Diwakar
Organization	EY (Ernst & Young)
Project	Internship GenAI Project
Date	April 2026

1. Executive Summary

The GenAI RAG System is a production-grade Retrieval-Augmented Generation platform developed during an internship at EY. It enables users to query documents stored in Google Drive through a conversational chat interface, receiving grounded, cited answers backed by real document content.

Beyond the core RAG engine, the system has been extended into a full product platform — KnowledgeVerse — which adds a multi-panel dashboard, real-time document search, user behavior tracking, chat history, and response caching on top of the underlying AI infrastructure.

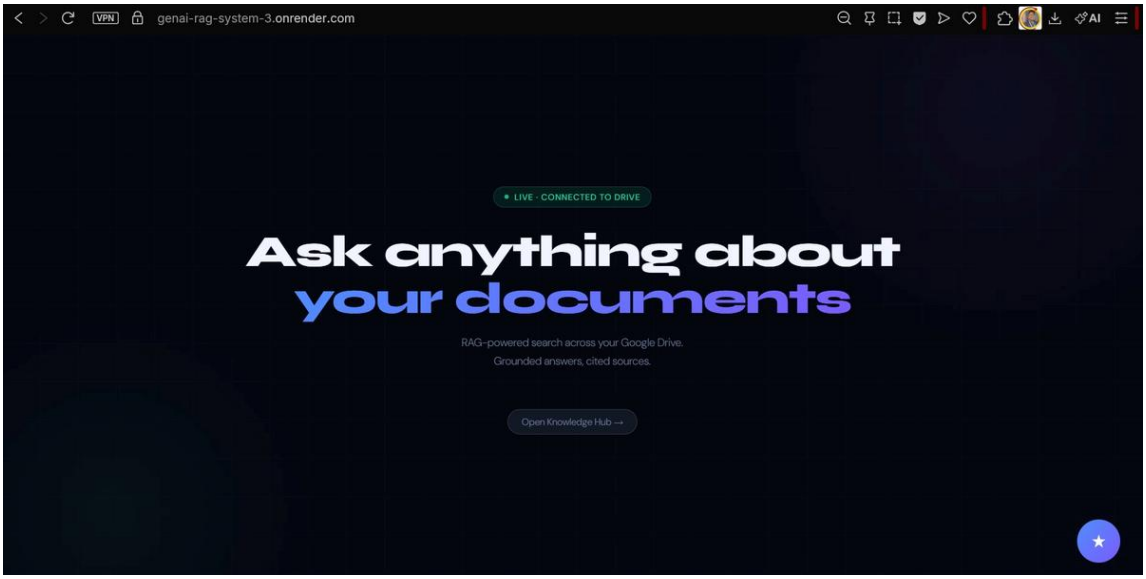

Figure 1: Live deployment — landing page

Attribute	Value
Deployment	Cloud-hosted on Render via Docker containers
Supported formats	PDF (text, scanned, chart-heavy, tables), DOCX, CSV
Retrieval strategy	Hybrid dense + BM25 + Reciprocal Rank Fusion (RRF)
LLM routing	Dynamic selection across Llama 3.3 70B, GPT-4o Mini, Claude Haiku
Automation	Daily document ingestion via GitHub Actions
Backup	Full index recovery from Google Drive
Live URL	https://genai-rag-system-3.onrender.com
Dashboard (NEW)	KnowledgeVerse — Latest Docs, Frequently Visited, Recent Activity
Search (NEW)	Real-time prefix-based document search with live dropdown

Attribute	Value
Analytics (NEW)	Click event logging, query tracking, multi-user activity feed
Chat History (NEW)	Date-grouped persistent query history with source attribution
Response Caching (NEW)	In-line CACHED indicator for repeated queries

2. Project Overview & Evolution

The system evolved from a simple local proof-of-concept into a fully cloud-native, containerised, and automated platform over the internship period (January 19, 2026 – April 2026).

2.1 Phase 1 — Foundation (Jan 20 – Jan 27, 2026)

Basic document ingestion from Google Docs into a local vector database; Streamlit chat interface; local LLM (Ollama/Mistral) for answer generation. ChromaDB as initial vector store.

2.2 Phase 2 — Chunking & Retrieval Quality (Feb 4 – Feb 14, 2026)

Migrated to paragraph-level chunking; upgraded to BGE-M3 embeddings; extended format support to DOCX and PDF; introduced CSV ingestion with deterministic Pandas computation engine; separated structured and semantic retrieval paths.

2.3 Phase 3 — Architecture Modernisation (Feb 27, 2026)

Migrated embeddings and LLMs to cloud-hosted APIs; introduced Gemini Vision OCR for scanned PDFs; modularised codebase into ingestion, data, query, and background job layers; integrated APScheduler for automated Drive sync.

2.4 Phase 4 — Production Deployment (Mar 5, 2026)

Deployed to Render Cloud via Docker; migrated vector store from local ChromaDB to Qdrant Cloud; replaced APScheduler with GitHub Actions for daily ingestion; implemented two-layer backup with boot-time restoration.

2.5 Phase 5 — Advanced Retrieval & Graph Execution (Mar–Apr 2026)

Added query rewriting, cross-encoder reranking (ms-marco-MiniLM-L-6-v2), LangGraph-based graph execution, multi-model LLM routing, and migrated frontend from Streamlit to React.js with FastAPI backend.

2.6 Phase 6 — Product Expansion Layer (Apr 2026)

NEW PHASE

Extended the platform beyond the core RAG engine into a full product experience. Added the KnowledgeVerse dashboard, real-time prefix search, user activity tracking, click event logging, chat history with date grouping, response caching with CACHED indicator, and Google Drive URL integration throughout the UI.

3. High-Level Architecture

The system is organised into five functional layers: Data Sources & Ingestion, Storage, Query & Retrieval, Frontend & API, and the new User Interaction & Analytics Layer. Each layer operates independently and communicates through well-defined interfaces.

3.1 Data Sources Layer

All documents are sourced from a configured Google Drive folder. The system supports PDF (text, scanned, chart-heavy, tables), DOCX, and CSV formats with format-aware extraction strategies.

Document Type	Extraction Method	Chunking Strategy
PDF (text-based)	pdfplumber	Balanced structural chunking
PDF (scanned / chart-heavy)	Gemini Vision OCR	Page-level extraction
PDF (embedded tables)	pdfplumber table parser	Row-aware contextual chunks
DOCX	python-docx	Paragraph-based chunks
CSV	Pandas	Direct SQLite storage

3.2 Ingestion Pipeline

Documents flow through the ingestion pipeline automatically via GitHub Actions on a daily schedule (3:00 AM IST). The pipeline is incremental — only new or updated files are processed.

3.3 Storage Layer

Two separate storage backends serve different data types. Qdrant Cloud hosts all vector embeddings for semantic search. SQLite (tracker.db) handles document metadata, CSV structured data, and now also stores latest document records, access event logs, and query activity logs.

3.4 Backup & Recovery Layer

SQLite is serialised, gzip-compressed, and uploaded to Google Drive as sqlite_latest.pkl.gz after every ingestion cycle. Qdrant creates a collection snapshot uploaded alongside. On Render startup, both are restored automatically, making the system query-ready within seconds of boot.

3.5 User Interaction & Analytics Layer

NEW LAYER

A behavioral data layer sits above the core RAG infrastructure and captures all user interactions. Document click events, query submissions, and search events are asynchronously logged to extended SQLite tables. These records power the dashboard's Frequently Visited and Recent Activity panels, creating a live multi-user activity feed.

4. KnowledgeVerse Dashboard

NEW SECTION

The KnowledgeVerse dashboard is the primary interface, acting as the entry point into the platform. It sits above the chat interface and provides three live data panels driven by real system state.

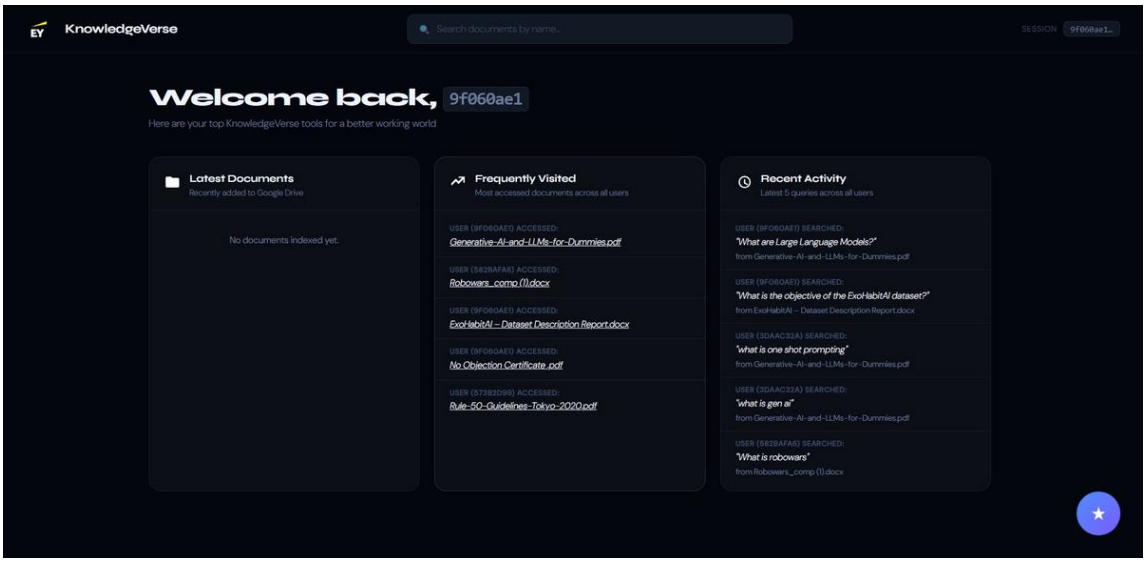

Figure 2: KnowledgeVerse dashboard — Latest Documents, Frequently Visited, and Recent Activity panels

4.1 Dashboard Panels

Panel	Data Source	Scope	Key Behaviour
Latest Documents	SQLite ingestion timestamp	Global (all users)	Shows 5 most recently ingested files with clickable Google Drive links. Auto-updates after each nightly ingestion cycle.
Frequently Visited	SQLite access_log table	Global (all users)	Lists each document access event with user session ID and clickable file link. Reflects real-time click behavior across all users.
Recent Activity	SQLite query_log table	Global (all users)	Shows latest 5 queries with query text, source document cited, and user session ID.

4.2 Real-Time Search Bar

The header search bar enables live document discovery. Each keystroke issues a request to the /search-documents endpoint, which performs a prefix match against all indexed file names and returns matching results as a dropdown. Selecting any result opens the corresponding Google Drive document in a new tab.

Component	Implementation
Search API	/search-documents?q={prefix} — prefix match against SQLite file name index
UI	Dropdown renders dynamically on each keystroke; dismissed on outside click
Click action	Each result maps to a Google Drive URL — opens document in new tab
Integration	File ID to Drive URL mapping stored in SQLite at ingestion time

4.3 Session Identity

Each browser session is assigned a unique session ID displayed in the dashboard header (e.g., 9F060AE1). This ID is used to attribute access events and query logs to individual users within the shared global feed, providing both personal traceability and cross-user visibility.

5. Query Processing Flow

When a user submits a question through the chat interface, the system executes a multi-stage pipeline to produce a grounded, cited answer. The pipeline is graph-based using LangGraph, allowing dynamic routing based on query type.

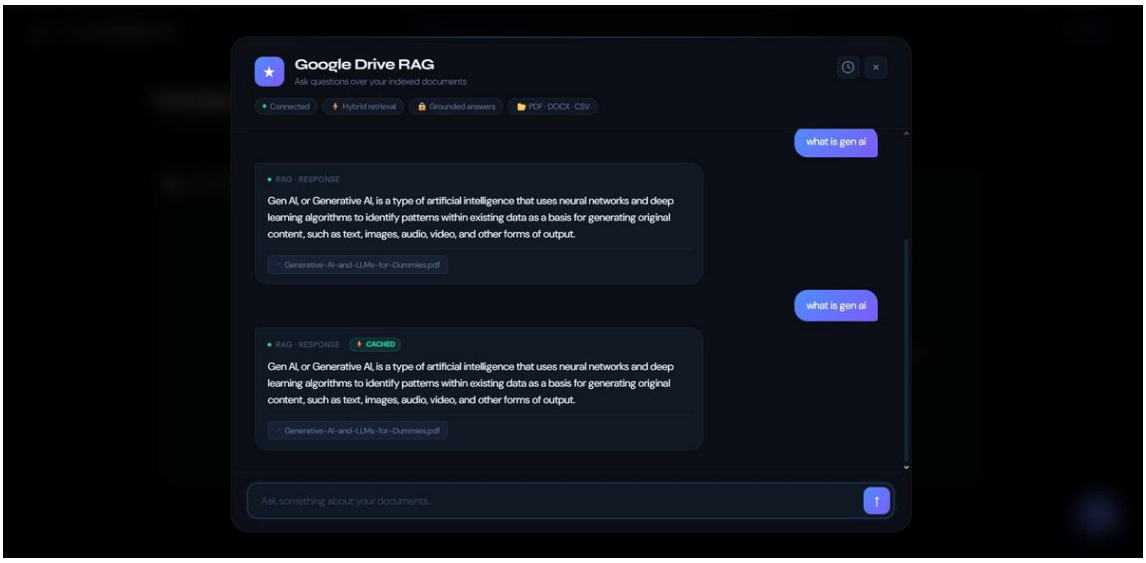

Figure 3: Google Drive RAG assistant — grounded answer with source attribution and CACHED response indicator

5.1 Query Classification

The first node classifies the incoming query as either structured (numerical/tabular CSV questions) or semantic (open-ended document questions). Classification uses intent detection to identify numeric operators, column references, and aggregation keywords.

5.2 Response Cache Check

NEW STEP

Before executing the full retrieval pipeline, the system checks whether an identical query has been answered previously. If a cached response exists, it is returned immediately with a CACHED badge in the UI, bypassing all retrieval and LLM steps.

5.3 Structured Path (CSV Queries)

Structured queries follow a deterministic computation path: intent is parsed, relevant columns are detected, Pandas computes the result from SQLite, and an LLM wraps the numeric result in a natural language explanation. No LLM is involved in the computation itself.

5.4 Semantic Path (Document Queries)

Semantic queries follow the full hybrid retrieval pipeline across seven steps:

Step	What Happens
1. Query Rewriting	Short or ambiguous queries are expanded by an LLM to improve recall by aligning with document vocabulary.
2. Hybrid Retrieval	Parallel dense vector search (Qdrant) and BM25 lexical search, fused via Reciprocal Rank Fusion (RRF).
3. Candidate Expansion	A large candidate pool (~40 chunks) is retrieved before final selection to maximise relevant chunk inclusion.
4. Cross-Encoder Reranking	ms-marco-MiniLM-L-6-v2 evaluates each (query, chunk) pair and reorders results by semantic relevance.
5. Additional Ranking Signals	Keyword overlap, exact phrase boosting, file name alignment, numeric token detection, structured/vision chunk priority, entity density scoring.
6. File Diversity Filter	Caps each file at 2 chunks in the final selection to prevent single-document dominance.
7. LLM Routing & Generation	The LLM router selects the appropriate model (Llama 3.3 70B for QA, GPT-4o Mini for summarisation, Claude Haiku for reasoning) and generates a grounded, cited answer.

5.5 Activity Logging

NEW STEP

After every query response is returned, the query text, source document, and session ID are asynchronously logged to the SQLite activity table. These records surface in the dashboard's Recent Activity panel, making all user queries visible across the shared feed.

6. Chat History

NEW SECTION

The RAG chat interface now includes a persistent chat history panel, accessible via the clock icon in the chat header. History is grouped by date (Today, Yesterday, and prior dates) and displays each query with its response summary and source document.

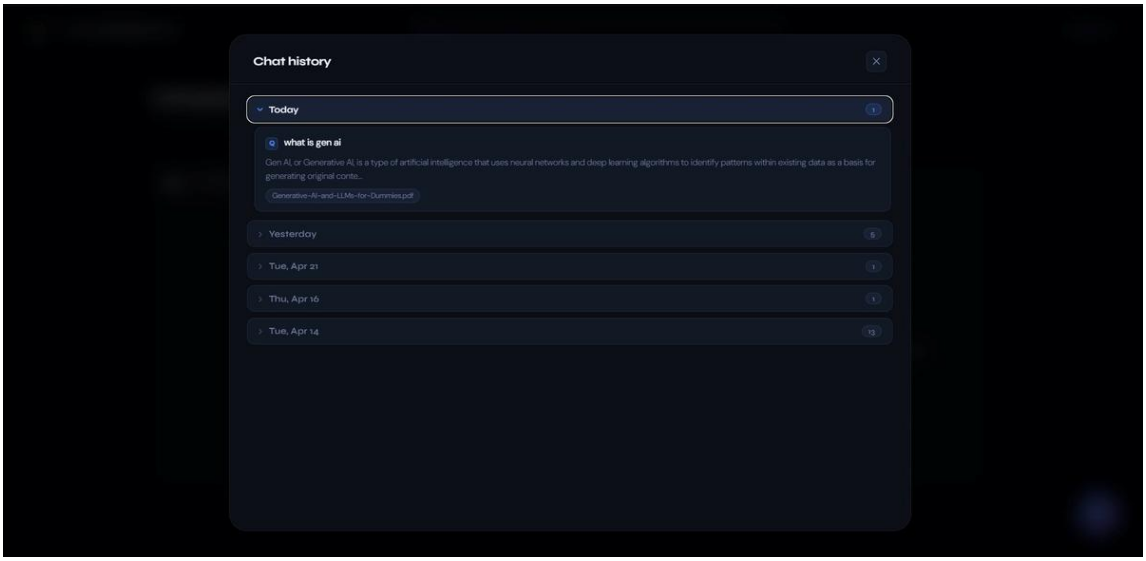

Figure 4: Chat history panel showing date-grouped query history with source document attribution

Feature	Detail
Grouping	Queries grouped by date: Today, Yesterday, and calendar dates for older entries
Entry content	Query text, truncated response preview, source document badge
Persistence	History persists across sessions for each user session ID
Entry count	Each date group shows count of queries for that day (e.g., badge showing '5')
Navigation	Collapsible accordion — expand any date to view its queries

7. Low-Level Architecture

7.1 Ingestion Engine — Component Breakdown

7.1.1 Google Drive Connector

Uses Google Drive API v3 with service account authentication. Performs incremental sync by comparing Drive file modification timestamps against SQLite tracker. Detects new, modified, and deleted files. Prevents duplicate ingestion through the tracker.

7.1.2 Format-Aware Parser

Format	Library	Strategy
PDF (text-based)	pdfplumber	Text extracted page-by-page, paragraph structure preserved
PDF (scanned / chart-heavy)	Gemini Vision Flash	Pages rasterised, OCR extracts text from images and diagrams
PDF (embedded tables)	pdfplumber table parser	Row and cell data extracted with row integrity preserved
DOCX	python-docx	Paragraph and heading structure preserved
CSV	Pandas	Column headers and rows stored directly in SQLite

7.1.3 Adaptive Chunking Engine

Chunk generation strategy is format-specific: PDF text uses balanced structural chunking with paragraph grouping; scanned PDFs are preserved as full-page single chunks; PDF tables use row-aware atomic chunks; DOCX uses paragraph-based chunks with heading hierarchy; CSV is stored directly in SQLite without chunking.

7.1.4 Embedding Generation

Parameter	Value
Model	BAAI/bge-small-en-v1.5
Provider	HuggingFace Inference Router
Vector dimensions	384
Normalisation	L2 normalised vectors

7.1.5 Latest Documents Update

NEW COMPONENT

After each ingestion cycle, the tracker.db latest_documents table is updated with file name, file ID, Drive URL, and ingestion timestamp for every newly indexed document. The dashboard's Latest Documents panel reads from this table and auto-refreshes to reflect newly ingested files.

7.2 Query Engine — Component Breakdown

7.2.1 LangGraph State Machine

The query engine is a LangGraph StateGraph. Execution flows through independent nodes connected by conditional edges, all sharing a RAGState object carrying the query, retrieved chunks, classification result, rewritten query, and final answer.

Node	Responsibility
Query Classification Node	Classifies query as structured or semantic
Cache Check Node (NEW)	Returns cached response if an identical query has been answered before
Query Rewriting Node	Expands the query using an LLM for better retrieval alignment
Retrieval Node	Executes hybrid dense + BM25 retrieval against Qdrant
Reranking Node	Applies cross-encoder scoring and additional ranking signals
Diversity Filter Node	Enforces per-file chunk caps
Generation Node	Routes to the appropriate LLM and constructs the prompt with context
Validation Node	Verifies the response is grounded in the provided context
Activity Log Node (NEW)	Logs the query, source document, and session ID to SQLite

7.2.2 Qdrant Vector Store Interface

All dense retrieval queries are issued to Qdrant Cloud using the Qdrant Python client. Each vector payload stores: file_id, file_name, chunk_id, document_type, page_number, chunk_text, is_vision_chunk, is_table_chunk, and drive_url.

7.2.3 Reciprocal Rank Fusion (RRF)

RRF merges ranked lists from dense and BM25 retrieval. Formula: RRF_score(chunk) = 1/(k + rank_dense) + 1/(k + rank_bm25), where k = 60. Chunks highly ranked in both lists receive the highest combined scores.

7.3 Analytics Engine

NEW SUBSYSTEM

Table	Schema	Written By	Read By
latest_documents	file_name, file_id, drive_url, ingested_at	Ingestion pipeline (nightly)	Dashboard /latest-documents endpoint
access_log	session_id, file_name, file_id, drive_url, accessed_at	Click event API (/track-access)	Dashboard /frequently-visited endpoint
query_log	session_id, query_text, source_document, answered_at	Query engine (post-generation)	Dashboard /recent-activity endpoint
response_cache	query_hash, response_text, source_document, created_at	Query engine (post-generation)	Cache Check Node at query time

7.4 Backend Architecture

The backend is a FastAPI service hosted on Render. Core endpoints handle query processing and document status. Extended endpoints support the dashboard, search, and analytics. LangGraph orchestrates the query execution graph with conditional node routing.

7.5 Frontend Architecture

The React.js single-page application renders the landing page, KnowledgeVerse dashboard (with search bar, session ID, and three data panels), Google Drive RAG chat interface (with message history, source attribution, and CACHED badges), and chat history modal (date-grouped, collapsible accordion).

8. Deployment Architecture

Task	Render (Docker)	GitHub Actions
Chat UI / FastAPI backend	Yes	No
Dashboard APIs (NEW)	Yes	No
Retrieval & generation	Yes	No
Response caching (NEW)	Yes	No
Qdrant Cloud connection	Yes	No
SQLite restoration on boot	Yes	No
Document ingestion	No	Yes
Embedding generation	No	Yes
Google Drive sync	No	Yes
Latest documents update (NEW)	No	Yes

8.1 Docker Container

The Docker image packages the FastAPI backend and React frontend into a single reproducible container, exposing the service on port 8501. On cold start, the container downloads and restores SQLite and Qdrant snapshots from Google Drive before accepting requests.

8.2 GitHub Actions Ingestion Workflow

Runs at 3:00 AM IST daily. Performs the complete ingestion cycle: Drive sync, extraction, embedding, Qdrant update, SQLite update (including latest_documents table), and backup upload to Google Drive. The running application is not restarted; new documents appear in the dashboard automatically after indexing completes.

8.3 Environment Variables

Variable	Purpose
GROQ_API_KEY	Groq LLM API access
OPENROUTER_API_KEY	OpenRouter LLM API access (GPT-4o Mini, Claude Haiku)
HF_API_KEY	HuggingFace Inference Router for embeddings
QDRANT_URL	Qdrant Cloud cluster endpoint URL
QDRANT_API_KEY	Qdrant Cloud authentication
GOOGLE_SERVICE_ACCOUNT_JSON	Google Drive access credentials (JSON)

9. Technical Stack

Layer	Component	Technology
Language Runtime	Python 3.11	Core backend language
Ingestion	PDF parsing	pdfplumber
Ingestion	Vision OCR	Gemini Vision Flash
Ingestion	DOCX parsing	python-docx
Ingestion	Structured processing	Pandas
Embeddings	Embedding model	BAAI/bge-small-en-v1.5
Embeddings	Inference provider	HuggingFace Inference Router
Storage	Vector store	Qdrant Cloud
Storage	Metadata / structured	SQLite (tracker.db)
Storage (NEW)	Analytics tables	SQLite extended (access_log, query_log, latest_documents, response_cache)
Retrieval	Sparse retrieval	BM25
Retrieval	Fusion algorithm	Reciprocal Rank Fusion (RRF)
Retrieval	Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2
LLM	Primary model	Llama 3.3 70B (Groq)
LLM	Summarisation	GPT-4o Mini (OpenRouter)
LLM	Reasoning	Claude Haiku (OpenRouter)
Orchestration	Graph execution	LangGraph
Backend	API framework	FastAPI
Frontend	UI framework	React.js
Frontend (NEW)	Dashboard	KnowledgeVerse — React components for search, analytics panels, session identity
Containerisation	Container runtime	Docker
Deployment	Cloud platform	Render
Automation	Scheduler	GitHub Actions
Backup	Storage	Google Drive

10. System Characteristics & Design Principles

10.1 Guiding Design Principles

Principle	Detail
No document-specific hardcoding	Retrieval is fully data-driven; no document names or content are hardcoded into the pipeline.
Deterministic structured reasoning	Numerical queries bypass LLM computation entirely, guaranteeing reproducible results.
Hybrid retrieval	Combining dense semantic search with BM25 improves both recall and ranking stability over pure dense approaches.
Cross-encoder semantic reranking	Post-retrieval reranking improves final chunk ordering beyond embedding similarity alone.
Query rewriting	Expands recall for short or ambiguous queries without requiring user reformulation.
Cloud-native design	Hosted embeddings, remote vector store, cloud backups, and API-based LLMs eliminate all local hardware dependencies.
Backup-safe operation	Full index recovery from Drive artifacts means the system never requires re-ingestion after a restart.
Shared global analytics state (NEW)	Dashboard panels operate on global shared state, surfacing cross-user usage patterns and creating a community activity feed.
Single-database analytics (NEW)	The existing SQLite tracker was extended with analytics tables rather than introducing a separate database, preserving the unified backup strategy.
Non-blocking interaction tracking (NEW)	Access events and query logs are written asynchronously so tracking never adds latency to the user-facing query response.

10.2 Known Limitations & Next Phase

Area	Current State	Next Phase
Observability	Application logs written locally within container. No centralised log aggregation.	Implement structured logging, distributed tracing, and a monitoring dashboard.
Query latency	Acceptable at current scale but not systematically profiled.	Profile each pipeline stage and optimise embedding, retrieval, and LLM routing latencies.
External integrations	Basic FastAPI endpoints. No external API gateway.	Build a versioned API gateway with authentication for webhook-based integrations.
Document sources	Google Drive only.	Evaluate Azure Blob Storage and SharePoint as alternative source layers.
Analytics depth (NEW)	Basic click and query tracking operational.	Use behavioral data to power document recommendations and auto-rank frequently accessed content.
Personalisation (NEW)	Dashboard shows global shared state.	Introduce per-user preferences and personalised recommendations based on individual history.

11. Glossary

Term	Definition
RAG	Retrieval-Augmented Generation — a technique that grounds LLM responses in retrieved document content.
BM25	Best Match 25 — a probabilistic lexical retrieval algorithm based on term frequency and inverse document frequency.
RRF	Reciprocal Rank Fusion — a rank combination algorithm that merges multiple ranked lists into a single unified ranking.
Cross-Encoder	A model that jointly encodes a (query, document) pair to produce a relevance score.
Dense Retrieval	Retrieval using vector embeddings and approximate nearest neighbor search.
LangGraph	A library for building graph-based LLM pipelines with stateful, conditional execution flows.
Qdrant	A cloud-native vector database optimised for high-performance approximate nearest neighbor search.
LLM Router	A component that dynamically selects the most appropriate language model based on query type and model availability.
Vision OCR	Optical Character Recognition using a vision model (Gemini) to extract text from images, scanned pages, or charts.
KnowledgeVerse (NEW)	The dashboard product layer of the platform, providing document discovery, activity feeds, and real-time search.
Access Log (NEW)	SQLite table recording document click events with session ID, file details, and timestamp. Powers the Frequently Visited dashboard panel.
Query Log (NEW)	SQLite table recording query submissions with session ID, query text, source document, and timestamp. Powers the Recent Activity panel.
Response Cache (NEW)	A query-level cache storing previously answered questions and their responses. Returns results instantly with a CACHED badge for repeated queries.
Session ID (NEW)	A unique identifier assigned to each browser session, used to attribute access events and query logs to individual users within the shared global activity feed.